

PyGameMaker Events & Actions Testing Checklist

This document tracks which events and actions have been tested and validated. The items are organized by game type priority, from simplest to most complex.

Testing Status Legend

- [] Not tested
- [x] Tested and working
- [!] Has known issues (see notes)
- [-] Not applicable / deprecated

NEW IN 1.0.0-rc.12 — Phase 2 runtime features

Cross-cutting runtime capabilities that apply across all game types below. Worth validating once before going through the per-game-type sections.

Bundled sample projects (Welcome tab → Try a sample game)

The bundled samples below double as ready-made fixtures for the per-game-type sections that follow. Each click opens a fresh editable copy under <Documents>/PyGameMaker Projects/; the bundled samples/<name>/ folder stays pristine (read-only by copy-on-open). Validate the click flow once, then reuse the same projects for the phase-specific tests.

Status	Sample	Useful for testing	Notes
[]	maze_1	Phase 1-2 (Sokoban / Labyrinth)	smallest, 2 rooms / 3 objects / 3 sprites
[]	maze_2	Phase 1-2	3 rooms / 9 objects / 7 sprites / 4 sounds
[]	maze_3	Phase 1-2	6 rooms / 17 objects / 16 sprites / 8 sounds
[]	maze.playground	Aseba playground export round-trip	XML file, opens via the Playground editor

(treasure and maze_4 were dropped from the bundled set after rc.12 testing surfaced significant GMK-import edge cases — see samples/README.md and TODO.md.)

For deeper per-sample passes during rc.12 testing, see docs/maze_1_testing_pass.md, docs/maze_2_testing_pass.md, and docs/maze_3_testing_pass.md.

For each sample, confirm: click in the Welcome tab opens immediately (no destination prompt), title bar shows the sample name, working copy lives under <Documents>/PyGameMaker Projects/, original samples/<name>/ is unchanged after Save.

Pixel-perfect collision — static, opt-in per sprite

Status	Item	Notes
[]	Sprite editor's "Precise Collision" checkbox toggles the per-sprite flag	shipped in rc.12; editors/sprite_editor
[]	Saving the project then reopening preserves the precise flag	
[]	At runtime, two precise=True sprites whose AABBs overlap but whose masks don't (e.g. circle near corner of square) → no collision event	static instances only
[]	Same setup but masks DO overlap → collision event fires	
[]	Rotated or non-unity-scaled instance with precise=True falls back to AABB	documented static-only limitation
[]	GMK import of an old project carries the per-sprite precise flag through	

Views / camera (single view — Phase 2b)

Status	Item	Notes
[]	New project with room.views_enabled = False renders identically to pre-rc.12 (regression guard)	
[]	Setting room.views_enabled = True + one visible view → only the view's port rect is drawn into	
[]	View's follow target stays inside (hborder, vborder) of the view edges as it moves	
[]	Follow shifts clamp to the room bounds (view doesn't go past the edge)	

Views / camera (multi-view / split-screen — Phase 2c)

Status	Item	Notes
[]	Two adjacent visible views with different follow targets → split-screen renders both correctly	
[]	Area between/around viewports shows the room background colour, not stale pixels	

[]	Per-view hspeed / vspeed clamps the per-frame camera movement; -1 is instant	
[]	room.current_view_index is set during render (queryable from draw events)	

PHASE 1: Sokoban-like Games

Grid-based puzzle games with push mechanics

Required Events

Status	Event	Description	Notes
[]	create	Object initialization	
[]	keyboard (held)	Arrow keys for movement	
[]	keyboard_no_key	Stop when no key pressed	
[]	collision	Push boxes, hit walls	
[]	step	Game logic per frame	

Required Actions - Movement

Status	Action	Description	Notes
[]	set_hspeed	Horizontal movement	
[]	set_vspeed	Vertical movement	
[]	stop_movement	Stop all movement	
[]	snap_to_grid	Align to grid cells	
[]	jump_to_position	Teleport (for pushing)	
[]	if_on_grid	Check grid alignment	

Required Actions - Control

Status	Action	Description	Notes
[]	if_collision	Check for obstacles	
[]	start_block	Begin conditional block	
[]	end_block	End conditional block	

Required Actions - Instance

Status	Action	Description	Notes
[]	destroy_instance	Remove objects	
[]	create_instance	Spawn objects	
[]	change_instance	Transform box types	

Required Actions - Room

Status	Action	Description	Notes
[]	next_room	Level progression	
[]	previous_room	Go back a level	
[]	restart_room	Reset puzzle	
[]	if_next_room_exists	Check for more levels	

PHASE 2: Labyrinth and Rogue-like Games

Maze exploration with enemies and items

Additional Events (beyond Phase 1)

Status	Event	Description	Notes
[]	keyboard_press	Single key actions	
[]	alarm_0 to alarm_3	Enemy AI timing	
[]	destroy	Death animations	

Additional Actions - Movement

Status	Action	Description	Notes
[]	start_moving_direction	8-way movement	
[]	set_direction_speed	Precise movement	
[]	move_towards_point	Enemy chase AI	
[]	check_keys_and_move	Grid movement helper	
[]	stop_if_no_keys	Grid movement helper	

Additional Actions - Control

Status	Action	Description	Notes
[]	test_chance	Random events	
[]	else_block	Alternative conditions	
[]	repeat	Multiple actions	
[]	exit_event	Stop event processing	
[]	set_variable	Track player stats	
[]	test_variable	Check player stats	

Additional Actions - Timing

Status	Action	Description	Notes
[]	set_alarm	Enemy spawn timers	

Additional Actions - Score/Lives/Health

Status	Action	Description	Notes
[]	set_health	Player health	
[]	test_health	Check if dead	
[]	draw_health_bar	Display health	
[]	set_lives	Extra lives	
[]	test_lives	Check game over	

Additional Actions - Audio

Status	Action	Description	Notes
[]	play_sound	Sound effects	
[]	stop_sound	Stop sounds	

PHASE 3: Platform Games

Side-scrolling with gravity and jumping

Additional Events (beyond Phase 2)

Status	Event	Description	Notes
[]	keyboard_release	Variable jump height	
[]	begin_step	Pre-movement logic	
[]	end_step	Post-movement logic	

Additional Actions - Movement

Status	Action	Description	Notes
[]	set_gravity	Apply gravity	
[]	set_friction	Deceleration	
[]	reverse_horizontal	Wall bounce	
[]	reverse_vertical	Ceiling bounce	
[]	jump_to_start	Respawn position	

[]	move_to_contact	Precise collision	
[]	bounce	Platform bounce	

Additional Actions - Control

Status	Action	Description	Notes
[]	if_object_exists	Check for enemies	
[]	test_instance_count	Win condition	

Additional Actions - Score

Status	Action	Description	Notes
[]	set_score	Point system	
[]	test_score	Score goals	
[]	draw_score	Display score	
[]	draw_lives	Display lives (icons)	

Additional Actions - Appearance

Status	Action	Description	Notes
[]	set_sprite	Animation states	
[]	transform_sprite	Flip direction	

PHASE 4: Scrolling Shooter Games

Vertical/horizontal shooters with projectiles

Additional Events (beyond Phase 3)

Status	Event	Description	Notes
[]	outside_room	Bullet cleanup	
[]	animation_end	Explosion finished	
[]	no_more_lives	Game over	
[]	no_more_health	Ship destroyed	

Additional Actions - Movement

Status	Action	Description	Notes
[]	jump_to_random	Enemy spawn points	
[]	wrap_around_room	Screen wrap	
[]	create_moving_instance	Fire projectiles	

Additional Actions - Instance

Status	Action	Description	Notes
[]	create_random_instance	Random powerups	
[]	destroy_at_position	Area damage	

Additional Actions - Appearance

Status	Action	Description	Notes
[]	set_color	Damage flash	

PHASE 5: Racing Games

Vehicle racing with acceleration and steering

Additional Events (beyond Phase 4)

Status	Event	Description	Notes

[]	intersect_boundary	Track boundaries	
[]	room_start	Race initialization	
[]	room_end	Race cleanup	

Additional Actions - Movement

Status	Action	Description	Notes
[]	(already covered in earlier phases)		

Additional Actions - Room

Status	Action	Description	Notes
[]	set_room_speed	Adjust game speed	
[]	set_room_caption	Display lap info	
[]	set_view	Camera follow	
[]	enable_views	Enable view system	

Additional Actions - Score

Status	Action	Description	Notes
[]	set_window_caption	Display time/lap	
[]	show_highscore	Best times	
[]	clear_highscore	Reset records	

PHASE 6: Zelda-like 2.5D RPG

Top-down action RPG with complex interactions

Additional Events (beyond Phase 5)

Status	Event	Description	Notes
[]	draw	Custom rendering	
[]	game_start	Game initialization	
[]	game_end	Save game state	
[]	mouse_* events	UI interaction	
[]	user_event_*	Custom triggers	

Additional Actions - Control

Status	Action	Description	Notes
[]	test_question	Dialogue choices	
[]	test_expression	Complex conditions	

Additional Actions - Room

Status	Action	Description	Notes
[]	goto_room	Specific room travel	
[]	check_room	Area-specific logic	
[]	set_room_persistent	Save room state	
[]	set_background_color	Theme changes	
[]	set_background	Area backgrounds	

Additional Actions - Appearance

Status	Action	Description	Notes
[]	draw_variable	Display stats	

Additional Actions - Audio

Status	Action	Description	Notes

[]	check_sound	Prevent overlap	
-----	-------------	-----------------	--

Complete Action Reference by Tab

Move Tab Actions

Status	Action	Priority
[]	start_moving_direction	Phase 2
[]	set_direction_speed	Phase 2
[]	move_towards_point	Phase 2
[]	set_hspeed	Phase 1
[]	set_vspeed	Phase 1
[]	set_gravity	Phase 3
[]	reverse_horizontal	Phase 3
[]	reverse_vertical	Phase 3
[]	set_friction	Phase 3
[]	jump_to_position	Phase 1
[]	jump_to_start	Phase 3
[]	jump_to_random	Phase 4
[]	snap_to_grid	Phase 1
[]	wrap_around_room	Phase 4
[]	move_to_contact	Phase 3
[]	bounce	Phase 3
[]	stop_movement	Phase 1
[]	if_on_grid	Phase 1
[]	stop_if_no_keys	Phase 2
[]	check_keys_and_move	Phase 2

Main1 Tab Actions (Instances)

Status	Action	Priority
[]	create_instance	Phase 1
[]	create_random_instance	Phase 4
[]	create_moving_instance	Phase 4
[]	change_instance	Phase 1
[]	destroy_instance	Phase 1
[]	destroy_at_position	Phase 4

Main2 Tab Actions (Appearance/Sound)

Status	Action	Priority
[]	set_sprite	Phase 3
[]	transform_sprite	Phase 3
[]	set_color	Phase 4
[]	play_sound	Phase 2
[]	stop_sound	Phase 2
[]	check_sound	Phase 6

Control Tab Actions

Status	Action	Priority
[]	if_collision	Phase 1
[]	if_object_exists	Phase 3
[]	test_instance_count	Phase 3
[]	test_chance	Phase 2

[]	test_question	Phase 6
[]	test_expression	Phase 6
[]	start_block	Phase 1
[]	end_block	Phase 1
[]	else_block	Phase 2
[]	repeat	Phase 2
[]	exit_event	Phase 2
[]	set_variable	Phase 2
[]	test_variable	Phase 2

Score Tab Actions

Status	Action	Priority
[]	set_score	Phase 3
[]	test_score	Phase 3
[]	draw_score	Phase 3
[]	show_highscore	Phase 5
[]	clear_highscore	Phase 5
[]	set_lives	Phase 2
[]	test_lives	Phase 2
[]	draw_lives	Phase 3
[]	set_health	Phase 2
[]	test_health	Phase 2
[]	draw_health_bar	Phase 2
[]	set_window_caption	Phase 5

Room/Extra Tab Actions

Status	Action	Priority
[]	previous_room	Phase 1
[]	next_room	Phase 1
[]	restart_room	Phase 1
[]	goto_room	Phase 6
[]	check_room	Phase 6
[]	if_next_room_exists	Phase 1
[]	if_previous_room_exists	Phase 1
[]	set_room_speed	Phase 5
[]	set_room_caption	Phase 5
[]	set_room_persistent	Phase 6
[]	set_background_color	Phase 6
[]	set_background	Phase 6
[]	enable_views	Phase 5
[]	set_view	Phase 5
[]	draw_variable	Phase 6

Timing Tab Actions

Status	Action	Priority
[]	set_alarm	Phase 2

Complete Event Reference

Create/Destroy Events

--	--	--

Status	Event	Priority
[]	create	Phase 1
[]	destroy	Phase 2

Step Events

Status	Event	Priority
[]	step	Phase 1
[]	begin_step	Phase 3
[]	end_step	Phase 3

Alarm Events

Status	Event	Priority
[]	alarm_0	Phase 2
[]	alarm_1	Phase 2
[]	alarm_2	Phase 2
[]	alarm_3	Phase 2
[]	alarm_4 to alarm_11	Phase 6

Keyboard Events

Status	Event	Priority
[]	keyboard (held)	Phase 1
[]	keyboard_press	Phase 2
[]	keyboard_release	Phase 3
[]	keyboard_no_key	Phase 1
[]	keyboard_any_key	Phase 6

Mouse Events

Status	Event	Priority
[]	mouse_left_button	Phase 6
[]	mouse_right_button	Phase 6
[]	mouse_left_press	Phase 6
[]	mouse_left_release	Phase 6
[]	Other mouse events	Phase 6

Collision Event

Status	Event	Priority
[]	collision	Phase 1

Other Events

Status	Event	Priority
[]	outside_room	Phase 4
[]	intersect_boundary	Phase 5
[]	animation_end	Phase 4
[]	no_more_lives	Phase 4
[]	no_more_health	Phase 4
[]	room_start	Phase 5
[]	room_end	Phase 5
[]	game_start	Phase 6
[]	game_end	Phase 6

Draw Event

--	--	--

Status	Event	Priority
[]	draw	Phase 6

Testing Notes

Test Project Requirements

For each phase, create a test project that validates all the events and actions listed:

1. **Phase 1 Test Project:** Simple Sokoban clone
2. **Phase 2 Test Project:** Rogue-like dungeon crawler
3. **Phase 3 Test Project:** Mario-style platformer
4. **Phase 4 Test Project:** Space shooter
5. **Phase 5 Test Project:** Top-down racing game
6. **Phase 6 Test Project:** Mini Zelda clone

Known Issues Log

Document any bugs found during testing here:

Date	Phase	Event/Action	Issue Description	Status

Blockly Block Testing Status

The following Blockly blocks correspond to the actions above:

Events (BLOCK_REGISTRY["Events"])

Status	Block Type	Action Mapping
[]	event_create	create
[]	event_step	step
[]	event_draw	draw
[]	event_destroy	destroy
[]	event_keyboard_nokey	keyboard_no_key
[]	event_keyboard_anykey	keyboard_any_key
[]	event_keyboard_held	keyboard
[]	event_keyboard_press	keyboard_press
[]	event_keyboard_release	keyboard_release
[]	event_mouse	mouse_* events
[]	event_collision	collision
[]	event_alarm	alarm_0 to alarm_11

Movement Blocks

Status	Block Type	Action Mapping
[]	move_set_hspeed	set_hspeed
[]	move_set_vspeed	set_vspeed
[]	move_stop	stop_movement
[]	move_direction	start_moving_direction
[]	move_towards	move_towards_point
[]	move_snap_to_grid	snap_to_grid
[]	move_jump_to	jump_to_position
[]	grid_stop_if_no_keys	stop_if_no_keys
[]	grid_check_keys_and_move	check_keys_and_move
[]	grid_if_on_grid	if_on_grid
[]	set_gravity	set_gravity
[]	set_friction	set_friction
[]	reverse_horizontal	reverse_horizontal

[]	reverse_vertical	reverse_vertical
-----	------------------	------------------

Instance Blocks

Status	Block Type	Action Mapping
[]	instance_destroy	destroy_instance (self)
[]	instance_destroy_other	destroy_instance (other)
[]	instance_create	create_instance

Room Blocks

Status	Block Type	Action Mapping
[]	room_goto_next	next_room
[]	room_goto_previous	previous_room
[]	room_restart	restart_room
[]	room_goto	goto_room
[]	room_if_next_exists	if_next_room_exists
[]	room_if_previous_exists	if_previous_room_exists

Score/Lives/Health Blocks

Status	Block Type	Action Mapping
[]	score_set	set_score
[]	score_add	set_score (relative)
[]	lives_set	set_lives
[]	lives_add	set_lives (relative)
[]	health_set	set_health
[]	health_add	set_health (relative)
[]	draw_score	draw_score
[]	draw_lives	draw_lives
[]	draw_health_bar	draw_health_bar

Other Blocks

Status	Block Type	Action Mapping
[]	set_alarm	set_alarm
[]	set_sprite	set_sprite
[]	sound_play	play_sound
[]	output_message	(display message)